



RAJALAKSHMI INSTITUTE OF TECHNOLOGY

Kuthambakkam, Chennai – 600124

**CS23431-DESIGN AND ANALYSIS OF ALGORITHMS
LAB MANUAL**

Name of the student :
Register Number :
DEPT. & SECTION :
LAB SEMESTER/YEAR : IV/II

CS23431	DESIGN AND ANALYSIS OF ALGORITHMS LABORATORY	L	T	P	C
		3	0	2	4
LIST OF EXPERIMENTS:					
Searching and Sorting Algorithms					
1. Implement Linear Search. Determine the time required to search for an element. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.					
2. Implement recursive Binary Search. Determine the time required to search an element. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.					
3. Given a text txt [0...n-1] and a pattern pat [0...m-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that n > m.					
4. Sort a given set of elements using the Insertion sort and Heap sort methods and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n.					
Graph Algorithms					
5. Develop a program to implement graph traversal using Breadth First Search					
6. Develop a program to implement graph traversal using Depth First Search					
7. From a given vertex in a weighted connected graph, develop a program to find the shortest paths to other vertices using Dijkstra’s algorithm.					
8. Find the minimum cost spanning tree of a given undirected graph using Prim’s algorithm.					
9. Implement Floyd’s algorithm for the All-Pairs- Shortest-Paths problem.					
10. Compute the transitive closure of a given directed graph using Warshall's algorithm.					
Algorithm Design Techniques					
11. Develop a program to find out the maximum and minimum numbers in a given list of n numbers using the divide and conquer technique.					
12. Implement Merge sort and Quick sort methods to sort an array of elements and determine the time required to sort. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n.					
State Space Search Algorithms					
13.Implement N Queens problem using Backtracking.					
Approximation Algorithms Randomized Algorithms					
14. Implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation.					
15. Implement randomized algorithms for finding the kth smallest number.					
TOTAL : 30 PERIODS					

SEARCHING AND SORTING ALGORITHMS

1. Implement Linear Search. Determine the time required to search for an element. Repeat the experiment for different values of n , the number of elements in the list to be searched and plot a graph of the time taken versus n .

AIM :

To write a C program to implement Linear Search. Determine the time required to search for an element. Repeat the experiment for different values of n , the number of elements in the list to be searched and plot a graph of the time taken versus n .

ALGORITHM:

LinearSearch(Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7.

Step 3: if $A[i] = x$ then go to step 6.

Step 4: Set i to $i + 1$.

Step 5: Go to Step 2.

Step 6: Print element x Found at index i and go to step 8.

Step 7: Print element not found

Step 8: Stop

RESULT:

Thus the C program for linear search algorithm was executed successfully and the output was verified.

2. Implement recursive Binary Search. Determine the time required to search an element. Repeat the experiment for different values of n , the number of elements in the list to be searched and plot a graph of the time taken versus n .

AIM :

To write a C program to implement recursive Binary Search. Determine the time required to search an element. Repeat the experiment for different values of n , the number of elements in the list to be searched and plot a graph of the time taken versus n .

ALGORITHM:

Step 1: Start

Step 2: Initialize $low \leftarrow 0$ and $high \leftarrow n - 1$.

Step 3: While $low \leq high$, do:

3.1 Compute $mid \leftarrow low + (high - low) / 2$

3.2 Compare x with $A[mid]$

Step 4: If $x == A[mid]$, then Return mid (element found)

Step 5: Else if $x > A[mid]$, then Set $low \leftarrow mid + 1$ (search right subarray)

Step 6: Else ($x < A[mid]$), then Set $high \leftarrow mid - 1$ (search left subarray)

Step 7: If the loop terminates without finding x , Return *element not found*.

Step 8: Stop

RESULT:

Thus the C program for Recursive Binary Search algorithm was executed successfully and the output was verified.

3. Given a text txt [0...n-1] and a pattern pat [0...m-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that $n > m$.

AIM :

To write a C program to given a text txt [0...n-1] and a pattern pat [0...m-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that $n > m$.

ALGORITHM:

Step 1: Initialize indices $i \leftarrow 0$ (for text txt) and $j \leftarrow 0$ (for pattern pat).

Step 2: Compare pat[j] with txt[i].

Step 3: If pat[j] == txt[i], then Increment $i \leftarrow i + 1$ and $j \leftarrow j + 1$.

Step 4: If $j == m$ (length of pattern), then Report that the pattern is found at index $i - j$,
Set $j \leftarrow lps[j - 1]$ and continue searching.

Step 5: If a mismatch occurs after j matches (pat[j] $\neq$ txt[i]), then:

5.1 If $j \neq 0$, set $j \leftarrow lps[j - 1]$
 (do not increment i)

5.2 If $j == 0$, then
 Increment $i \leftarrow i + 1$

Step 6: Repeat Steps 2–5 until i reaches the end of the text.

Step 7: Stop.

RESULT:

Thus the C program for Pattern search algorithm was executed successfully and the output was verified.

4. Sort a given set of elements using the Insertion sort and Heap sort methods and determine the time required to sort the elements. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n .

AIM :

To write a c program to sort a given set of elements using the Insertion sort and Heap sort methods and determine the time required to sort the elements. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n .

ALGORITHM(INSERTION SORT):

Step 1: Start the program

Step 2: Iterate from $arr[1]$ to $arr[N]$ over the array.

Step 3: Compare the current element (key) to its predecessor.

Step 4: If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

Step 5: Repeat Steps 2–4 until the entire array is sorted.

Step 6: Stop the program.

ALGORITHM(HEAP SORT):

Step 1: Start the program

Step 2: Build a max heap from the input data.

Step 3: At this point, the maximum element is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of the heap by 1. Finally, heapify the root of the tree.

Step 4: Repeat step 2 while the size of the heap is greater than 1.

Step 5: Stop the program

RESULT:

Thus the c programs for Insertion and Heap Sort algorithms were executed successfully and the output was verified.

GRAPH ALGORITHMS

5. Develop a program to implement graph traversal using Breadth First Search.

AIM :

To write a c program to implement graph traversal using Breadth First Search.

ALGORITHM:

Step 1: Consider the graph you want to navigate.

Step 2: Select any vertex in your graph, say v1, from which you want to traverse the graph.

Step 3: Examine any two data structure for traversing the graph Visited array (size of the graph)

Queue data structure

Step 4: Starting from the vertex, you will add to the visited array, and afterward, you will v1's adjacent vertices to the queue data structure.

Step 5: Now, using the FIFO concept, you must remove the element from the queue, put it into the visited array, and then return to the queue to add the adjacent vertices of the removed element.

Step 6: Repeat step 5 until the queue is not empty and no vertex left to be visited.

RESULT:

Thus the c program for graph traversal using Breadth First Search algorithm was executed successfully and the output was verified.

6. Develop a program to implement graph traversal using Depth First Search.

AIM :

To write a C program to implement graph traversal using Depth First Search.

ALGORITHM:

Step 1: Start by putting any one of the graph's vertices at the back of the queue.

Step 2: Now take the front item of the queue and add it to the visited list.

Step 3: Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.

Step 4: Keep continuing steps two and three till the queue is empty.

Step 5: Stop the program

RESULT:

Thus the C program for Depth first search algorithm was executed successfully and the output was verified.

7. From a given vertex in a weighted connected graph, develop a program to find the shortest paths to other vertices using Dijkstra's algorithm.

AIM :

To write a c program from a given vertex in a weighted connected graph, to find the shortest paths to other vertices using Dijkstra's algorithm.

ALGORITHM:

Step 1: Mark the source node with a current distance of 0 and the rest with infinity.

Step 2: Set the non-visited node with the smallest current distance as the current node.

Step 3: For each neighbor, N of the current node adds the current distance of the adjacent node with the weight of the edge connecting 0→1. If it is smaller than the current distance of Node, set it as the new current distance of N.

Step 4: Mark the current node 1 as visited.

RESULT:

Thus the C program to find the shortest paths to other vertices using Dijkstra's algorithm was executed successfully and the output was verified.

8. Find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

AIM :

To write a C program to find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

ALGORITHM:

Step 1: Determine an arbitrary vertex as the starting vertex of the MST.

Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

Step 5: Add the chosen edge to the MST if it does not form any cycle.

Step 6: Return the MST and exit.

RESULT:

Thus the C program to find the minimum cost spanning tree of a given undirected graph using Prim's algorithm was executed successfully and the output was verified.

9. Implement Floyd's algorithm for the All-Pairs-Shortest-Paths problem.

AIM :

To write a C program to implement Floyd's algorithm for the All-Pairs-Shortest-Paths problem.

ALGORITHM:

Step 1: Initialize the distance (solution) matrix `dist` such that `dist[i][j]` is equal to the weight of the edge from vertex `i` to vertex `j`. If there is no direct edge, set `dist[i][j] = ∞`. Set `dist[i][i] = 0` for all vertices.

Step 2: Consider each vertex one by one as an intermediate vertex.

Step 3: For each vertex `k` from 0 to `n - 1`, do:

Assume vertices `{0, 1, 2, ..., k - 1}` have already been considered as intermediate vertices.

Step 4: For every pair of vertices `(i, j)`, update the distance as follows:

4.1 If vertex `k` is **not** an intermediate vertex in the shortest path from `i` to `j`, keep `dist[i][j]` unchanged.

4.2 If vertex `k` **is** an intermediate vertex in the shortest path from `i` to `j`, update

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

Step 5: Repeat Steps 3 and 4 until all vertices are considered as intermediate vertices.

Step 6: The final matrix `dist` contains the shortest distances between all pairs of vertices.

Step 7: Stop.

RESULT:

Thus the C program for Floyd's algorithm was executed successfully and the output was verified.

10. Compute the transitive closure of a given directed graph using Warshall's algorithm

AIM :

To write a C program to compute the transitive closure of a given directed graph using Warshall's algorithm.

ALGORITHM:

Step 1: Start

Step 2: Let the input graph be represented by an adjacency matrix $G[V][V]$.

Step 3: Initialize a Boolean matrix $reach[V][V]$ such that:

- $reach[i][j] = 1$ if there is a direct edge from vertex i to vertex j
- $reach[i][j] = 0$ otherwise
- Set $reach[i][i] = 1$ for all vertices

Step 4: For each vertex k from 0 to $V - 1$, do:

For each vertex i from 0 to $V - 1$, do:

For each vertex j from 0 to $V - 1$, do:

$reach[i][j] = reach[i][j] || (reach[i][k] \& \& reach[k][j])$

Step 5: After completion of all iterations, the matrix $reach$ represents the transitive closure of the graph.

Step 6: Stop

RESULT:

Thus the C program for Warshall's algorithm was executed successfully and the output was verified.

ALGORITHM DESIGN TECHNIQUES

11. Develop a program to find out the maximum and minimum numbers in a given list of n numbers using the divide and conquer technique.

AIM :

To write a C program to develop a program to find out the maximum and minimum numbers in a given list of n numbers using the divide and conquer technique.

ALGORITHM:

Step 1: Start the program.

Step 2: Divide array by calculating mid index i.e. $\text{mid} = l + (r - l)/2$.

Step 3: Recursively find the maximum and minimum of left part by calling the same function i.e.

$\text{leftMinMax}[2] = \text{minMax}(X, l, \text{mid})$

Step 4: Recursively find the maximum and minimum for right part by calling the same function i.e.

$\text{rightMinMax}[2] = \text{minMax}(X, \text{mid} + 1, r)$

Step 5: Finally, get the overall maximum and minimum by comparing the min and max of both halves.

$\text{min} = \min(\text{leftMin}, \text{rightMin})$

$\text{max} = \max(\text{leftMax}, \text{rightMax})$

Step 6: Return (min, max).

Step 7: Stop.

RESULT:

Thus the C program for finding maximum and minimum numbers in a given list of n numbers using the divide and conquer technique was executed successfully and the output was verified.

12. Implement Merge sort and Quick sort methods to sort an array of elements and determine the time required to sort. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n .

AIM :

To write a C program to implement Merge sort and Quick sort methods to sort an array of elements and determine the time required to sort. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph of the time taken versus n .

ALGORITHM (MERGE SORT):

Step 1: Start

Step 2: Read the number of elements n

Step 3: Read the array elements $A[0...n-1]$

Step 4: Record the starting time

Step 5: If the array has more than one element, do the following:

- a) Divide the array into two halves
- b) Recursively apply Merge Sort on the left half
- c) Recursively apply Merge Sort on the right half

Step 6: Merge the two sorted halves as follows:

- a) Compare elements from both halves
- b) Copy the smaller element into a temporary array
- c) Repeat until one half is exhausted
- d) Copy remaining elements of the other half
- e) Copy the temporary array back into the original array

Step 7: Record the ending time

Step 8: Calculate time taken = ending time - starting time

Step 9: Display the sorted array

Step 10: Display the time taken

Step 11: Stop

ALGORITHM (QUICK SORT):

Step 1: Start

Step 2: Read the number of elements n

Step 3: Read the array elements $A[0...n-1]$

Step 4: Record the starting time

Step 5: If the array has more than one element, perform the following steps:

- a) Select the last element as the pivot
- b) Rearrange the array such that elements smaller than or equal to the pivot are placed to its left and elements greater than the pivot are placed to its right
- c) Place the pivot in its correct position

Step 6: Recursively apply Quick Sort on the left subarray

Step 7: Recursively apply Quick Sort on the right subarray

Step 8: Repeat steps 5–7 until all subarrays contain only one element

Step 9: Record the ending time

Step 10: Calculate time taken = ending time - starting time

Step 11: Display the sorted array

Step 12: Display the time taken

Step 13: Stop

RESULT:

Thus the C program to sort an array of elements using Merge sort and Quick sort Algorithms were executed successfully and the output was verified.

13. Implement N Queens problem using Backtracking

AIM :

To write a C program to implement N Queens problem using Backtracking.

ALGORITHM:

Step 1: Initialize an empty chessboard of size $N \times N$.

Step 2: Start with the leftmost column and place a queen in the first row of that column.

Step 3: Move to the next column and place a queen in the first row of that column.

Step 4: Repeat step 3 until either all N queens have been placed or it is impossible to place a queen in the current column without violating the rules of the problem.

Step 5: If all N queens have been placed, print the solution.

Step 6: If it is not possible to place a queen in the current column without violating the rules of the problem, backtrack to the previous column.

Step 7: Remove the queen from the previous column and move it down one row.

Step 8: Repeat steps 4–7 until all possible configurations have been tried.

RESULT:

Thus the C program to implement N Queens problem using Backtracking was executed successfully and the output was verified.

APPROXIMATION ALGORITHMS & RANDOMIZED ALGORITHMS

14. Implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation.

AIM :

To write a C program to implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation.

ALGORITHM:

Step 1: Start

Step 2: Choose an arbitrary vertex as the starting vertex and mark it as visited. Set it as the current vertex.

Step 3: While there exists at least one unvisited vertex, do:

3.1 Find the unvisited vertex v that is nearest to the current vertex

3.2 Move to vertex v

3.3 Mark vertex v as visited

3.4 Set v as the current vertex

Step 4: Return to the starting vertex to complete the tour.

Step 5: The sequence of visited vertices represents the approximate TSP tour.

Step 6: Stop

RESULT:

Thus the C program problem to find the optimal solution for the Traveling Salesperson problem was executed successfully and the output was verified.

15. Implement randomized algorithms for finding the kth smallest number.

AIM :

To write a C program to implement randomized algorithms for finding the kth smallest number.

ALGORITHM:

Step 1: Find the k^{th} smallest element in array A using a **selection algorithm**.

Let the k^{th} smallest element be z.

Step 2: Initialize an empty list L.

Step 3: Initialize a counter $\text{count} \leftarrow 0$.

Step 4: For each element x in array A do

4.1 If $x < z$ then

4.1.1 Add x to list L

4.1.2 $\text{count} \leftarrow \text{count} + 1$

Step 5: For each element x in array A do

5.1 If $x = z$ **and** $\text{count} < k$ then

5.1.1 Add x to list L

5.1.2 $\text{count} \leftarrow \text{count} + 1$

Step 6: Return list L.

RESULT:

Thus the C program to implement randomized algorithm for finding the kth smallest number was executed successfully and the output was verified.